CLOUDWORX

Frontend Architecture Design Document

Part A · Frontend Developer (React) assessment

Prepared by	Pavithravignesh Sathasivam
Version	1.0
Date	20 October 2026
Part B	github.com/Pavithravignesh/3d-layout-editor
Live demo	3d-layout-editor.vercel.app

Contents

0	Summary	the decisions on one page
1	Context and design drivers	from the brief's numbers to design rules
2	Architecture overview (Deliverable A1)	diagram + protocol on every connection
3	Application structure	brief item 1
4	State and data	brief item 2
5	Grids and forms	brief item 3
6	Localisation and accessibility	brief item 4
7	Embedding in the host	brief item 5
8	The 3D editing workspace	brief item 6
9	3D asset optimisation	brief item 7
10	AI assistant panel	brief item 8
11	Performance and quality	brief item 9
12	End-to-end sequence: fix a clearance violation	Deliverable A2
13	Decision log and rejected trade-offs	Deliverable A2
14	Assumptions	Deliverable A2
15	Risks and open questions	
A	Appendix: Part B reference build, wireframes, glossary, use of AI tools	

> **Built, not only designed.** The core of Surface 2 (edit model, undo/redo, snapping, clearance rule, list/properties sync, instanced rendering, EN/VI) is implemented in Part B. Sections 8 and 11 quote its measured numbers, and Appendix A maps each section to the code.

0 Summary

One frontend team, two products. Surface 1 is a daily-use procurement app for buyers, suppliers and approvers in India, Vietnam and China on 8 GB laptops over VPN. Surface 2 is a 3D layout editor that runs inside another company's application on integrated graphics. The design optimises for the same things in both: **fast on a slow network, never losing work, equally good in three languages and without a mouse, and safe at the boundaries it does not own.**

≤ 2.5 s
first useful screen on 10 Mbps VPN · budgeted at ~1.9 s; Part B's first download is 34 KB

60 fps
500 items at 1080p on Intel Iris Xe in Part B, with 15 draw calls (budget: ≥ 50 fps, < 300 calls)

4.8 ms
clearance check for 2,000 items (spatial hash; 342 ms brute force)

40 MB → ≤ 0.65 MB
per CAD model after LODs, Meshopt and KTX2

The seven decisions that shape everything

Decision	Why
One Nx monorepo, versioned shared packages	Both surfaces build on `@cloudworx/ui`, tokens, i18n and API types, published with semantic versions. Each app pins its versions and deploys alone: shared code, separate release trains.
The server owns the data, the client caches it	TanStack Query is the client's single copy of server data. REST loads it, the WebSocket keeps it fresh, optimistic updates make it feel instant, and Zustand holds only small UI state.
Big grids stay on the server	AG Grid's server-side row model asks for 100-row blocks; filtering, sorting, search and bulk actions run on the server, so 200,000 rows cost the browser about the same as 2,000.
Nothing is lost on a flaky VPN	Drafts go to IndexedDB at once and autosave to the server; retries carry idempotency keys; the WebSocket resumes from its last sequence number; the UI says clearly when it is offline.
Surface 2 lives in an iframe behind a typed contract	The host keeps sign-in and navigation; the editor gets context and a short-lived token over an origin-checked `postMessage` contract, and takes the host's look from its brand tokens.
Every edit is a command	3D drags, panel edits, keyboard shortcuts and accepted AI proposals all become the same undoable commands in one store, so the 3D view, list and properties never disagree.
3D cost is fixed by design, not by luck	CAD models go through a pipeline (3 LODs, Meshopt, KTX2, per-model budgets) and are drawn with instancing, so draw calls grow with the number of *distinct models*, not items.

1 Context and design drivers

Each number in the brief becomes a rule the design must satisfy. The rest of the document refers back to these rules.

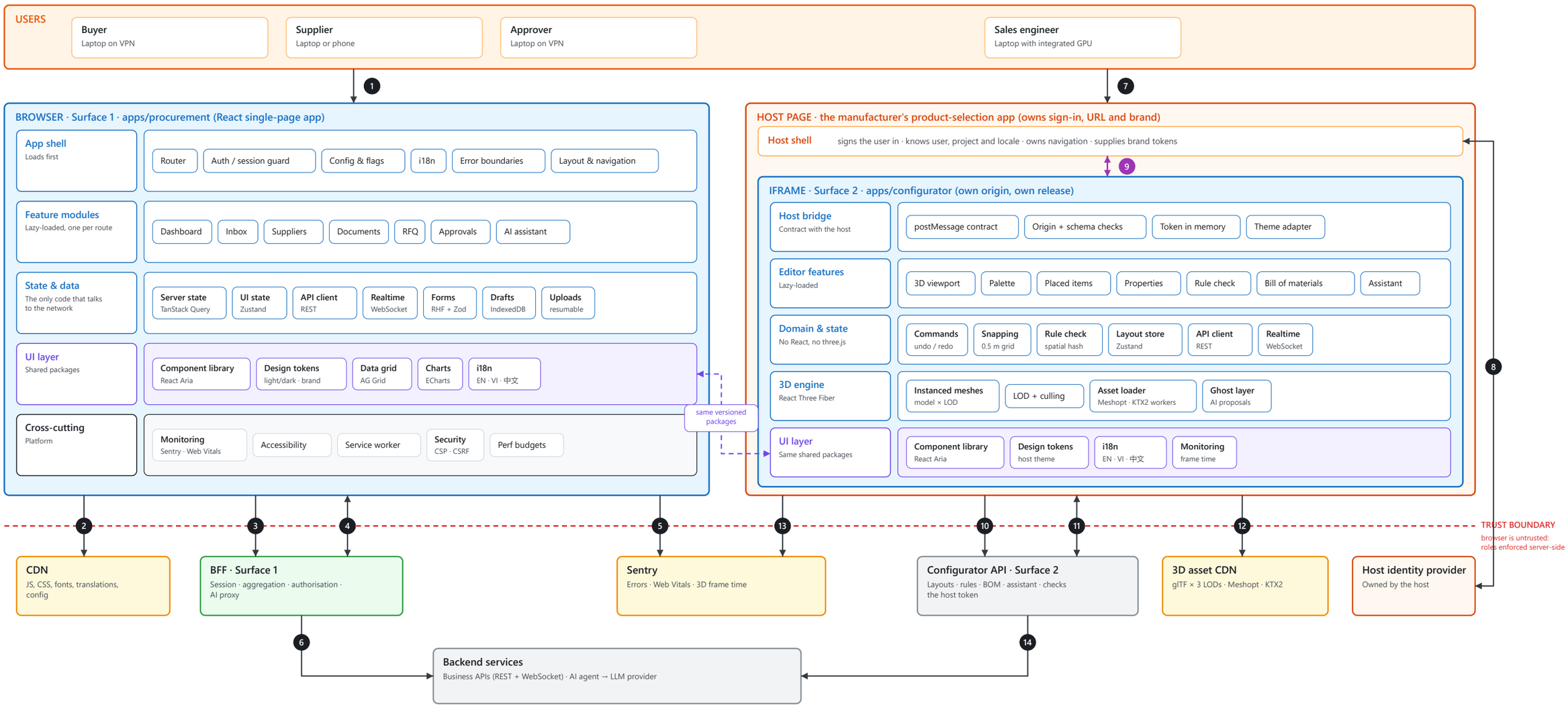
From the brief	What it implies	Design rule
3,000 buyer and supplier users, used all day	Long sessions, many tabs, repeat visits	Cache the app shell and translations; keep memory flat over a working day
400 sales engineers in the pilot, 5,000 within a year	12× growth on Surface 2	Stateless frontend; assets on a CDN; nothing in the client that assumes a small user count
8 GB laptops, integrated graphics, locked-down Chrome/Edge, VPN or VDI	Weak CPU and GPU, no extensions, high latency	Small initial JavaScript, zero-runtime CSS, fewer round trips, a 3D frame budget sized for integrated GPUs
Some suppliers on mid-range Android phones	Narrow screens, mobile data	Inbox, uploads, onboarding and quote submission work at 360 px; analysis screens are desktop-first
Grids of 50,000 suppliers or 200,000 documents	Too big to download or render	Server-side filtering, sorting, search and paging; virtualised rows; bulk actions sent as a filter
RFQs: 40 line items × 8 suppliers	Wide, dense, mixed currencies	Pinned headers, supplier columns generated from data, converted currency with the original kept
English, Vietnamese, Simplified Chinese from one build; VND, INR, CNY, USD	Runtime language switch, locale formats	Lazy translations per language and screen; `Intl` for every date, number and amount
WCAG 2.2 AA	Keyboard and screen-reader access everywhere, including 3D	Accessible primitives, axe in CI, scripted keyboard paths, a text twin of the 3D scene
Up to 2,000 placed items from 300 glTF models of 2–40 MB with three LODs	Raw CAD is unusable in a browser	An asset pipeline with per-model budgets; instancing by model and LOD (Section 9)
≥ 50 fps at 1080p on integrated graphics, < 300 draw calls, < 1 GB	20 ms per frame, everything included	A frame, draw-call and memory budget measured in CI and in the field (Section 11)
First useful screen ≤ 2.5 s on 10 Mbps VPN	Every kilobyte and round trip counts	A budget per resource, measured in CI on a throttled profile (Section 11)
Surface 2 inside the host page, host passes context, host design language	Someone else's page, sign-in and brand	Iframe with a typed, origin-checked contract; host tokens mapped onto ours (Section 7)
REST + WebSocket backends (fixed)	Two channels only	REST for reads and commands; the WebSocket for live updates *and* streamed AI replies; no third protocol

2 Architecture overview · Deliverable A1

Both surfaces drawn in layers inside the browser, the server side kept to a few boxes, the trust boundary, and the protocol on every connection (table in the diagram). Editable source: `diagram-a1.excalidraw`. Build and release: end of Section 3.

A1 · Frontend architecture: Surface 1 and Surface 2

Each app is drawn in layers; a layer uses only the layers below it. Both apps are built from the same versioned packages but released on their own. The server side is kept to a few boxes.



LEGEND

- Our apps (browser)
- Owned by the host
- Backend (internal)
- Shared, versioned packages
- Backend-for-frontend
- Third-party / delivery
- postMessage between host page and iframe
- trust boundary

Build and release are shown in the separate build-and-release diagram (Section 3).

#	From → To	Protocol
1	Users → Surface 1	HTTPS
2	Surface 1 → CDN	HTTPS GET, immutable cache
3	Surface 1 → BFF	HTTPS REST/JSON, session cookie
4	Surface 1 ↔ BFF	WebSocket: live updates, AI tokens
5	Surface 1 → Sentry	HTTPS
6	BFF → Backend	REST, internal (mTLS)
7	Sales engineer → host page	HTTPS

#	From → To	Protocol
8	Host shell ↔ host identity	OIDC (owned by the host)
9	Host page ↔ Surface 2 iframe	postMessage, origin-checked
10	Surface 2 → Configurator API	HTTPS REST/JSON, Bearer token
11	Surface 2 ↔ Configurator API	WebSocket: rule results, AI tokens
12	Surface 2 → 3D asset CDN	HTTPS GET, HTTP/2, cached
13	Surface 2 → Sentry	HTTPS
14	Configurator API → Backend	REST, internal (mTLS)

Connections and protocols

#	Connection	Protocol	What travels and why
①	Users → Surface 1	HTTPS (TLS 1.2+) over VPN/VDI	One build serves all roles and languages
②	Surface 1 → CDN	HTTPS GET, HTTP/2, immutable cache	Hashed JS/CSS, translations per language and screen, font subsets, runtime `config.json`
③	Surface 1 → BFF	HTTPS REST/JSON, session cookie	All reads and commands. Cookie `HttpOnly; Secure; SameSite=Lax`; headers: CSRF token, `X-Correlation-Id`, `Accept-Language`, `If-Match`, `Idempotency-Key`
④	Surface 1 ↔ BFF	WSS, JSON `{ seq, type, data }`	Task, quote, approval and job events plus AI reply tokens; heartbeat; resume from last `seq`. Session cookie; the BFF checks the `Origin` header on the handshake
⑤	Surface 1 → Sentry	HTTPS	Errors, traces and Web Vitals, personal data scrubbed
⑥	BFF → backend services	HTTPS REST, mTLS (internal)	Aggregation per screen; the BFF subscribes to the backend's WebSocket channel and filters events per user
⑦	Sales engineer → host page	HTTPS	The host's own application and sign-in
⑧	Host shell ↔ host identity provider	OIDC (owned by the host)	Issues a short-lived access token whose audience is the Configurator API
⑨	Host page ↔ Surface 2 iframe	`postMessage`, typed and origin-checked	Context, token refresh, theme, resize, navigation, errors (Section 7.2)
⑩	Surface 2 → Configurator API	HTTPS REST/JSON, `Authorization: Bearer`	Layouts, catalogue, layout operations with version, assistant runs
⑪	Surface 2 ↔ Configurator API	WSS, JSON events	Authoritative rule results, collaborator changes, `assistant.delta` tokens. Opened with a one-time ticket (Section 7.3)
⑫	Surface 2 → 3D asset CDN	HTTPS GET, HTTP/2, immutable cache	glTF per model and LOD, KTX2 textures, model manifest (Section 9)
⑬	Surface 2 → Sentry	HTTPS	Errors, WebGL context loss, frame time, draw calls and memory per device class
⑭	Configurator API → backend services	HTTPS REST, mTLS (internal)	Catalogue, pricing, AI agent; the agent calls the LLM provider server-side only

3 Application structure

How the two surfaces share a design system, tokens and components without sharing one release train, where the module boundaries are, and why this beats micro-frontends here.

3.1 One repository, separately versioned packages

```
cloudworx-frontend/                Nx + pnpm workspaces
├── packages/                      SHARED · published with semantic versions
│   ├── tokens/        1.4.0       design tokens → CSS variables
│   ├── ui/            2.3.0       React components (React Aria + CSS Modules)
│   ├── i18n/          1.1.0       i18next setup, ICU, Intl formatters, locale resolver
│   ├── api-types/     3.0.0       types generated from the OpenAPI specs
│   └── host-bridge/   1.0.0       typed postMessage contract (Surface 2 and test host)
├── apps/                          PRODUCTS · each deployed on its own
│   ├── procurement/               Surface 1
│   ├── configurator/              Surface 2 (embedded)
│   └── test-host/                 stands in for the host page in CI and staging
└── tooling/                       shared lint, TypeScript and test configs
```

One repository makes cross-cutting changes (a token and the component that uses it) a single reviewed change. Publishing the packages with versions keeps the **releases** separate.

3.2 How sharing works without a shared release train

1. A change to `packages/ui` merges with a **changeset** describing it as a patch, minor or major change.
2. CI publishes the new version (for example `@cloudworx/ui@2.4.0`) to the private registry with a changelog.
3. Each app pins the versions it uses. Surface 1 can take 2.4.0 this week while Surface 2 stays on 2.1.0 mid-release; nothing breaks for an app that did not opt in.
4. Nx `affected` builds, tests and deploys only the apps whose code or pinned versions changed. 3D models have their own pipeline (Section 9).

> Within the monorepo, apps consume packages through **pinned published versions**, not live workspace links, in release builds. Developers can link locally to try an unreleased change, but CI builds exactly what will ship.

3.3 Design tokens in layers

Layer	Example	Who changes it
Primitive	`--blue-600: #1971c2`	Design system team
Brand	`--brand-primary: var(--blue-600)`	Swapped per brand. Surface 2 maps the host's brand here at runtime (Section 7.7)
Semantic	`--color-action`, `--text-strong`, `--surface-raised`	The only layer components may use
Mode	Light / dark; compact / comfortable density	Set by an attribute on `<html>`; grids default to compact

Tokens are authored in Figma (Tokens Studio), exported as JSON and compiled by Style Dictionary into CSS variables and TypeScript constants. Components are styled with **CSS Modules** (scoped, zero runtime), read only

semantic variables, and use **CSS logical properties**, so a right-to-left language later needs no redesign. Part B's `src/design-system/` is a small working version of this package.

3.4 Module boundaries

Rule 1 · Dependencies point one way

Apps → packages → tokens; inside an app, `app` → `features` → `state` / `scene` → `domain`. Packages never import apps; apps never import each other.

Rule 2 · Each module has one front door

Every package and feature exports a public `index.ts`. Deep imports into another module's internals are lint errors, and `package.json` `exports` makes them fail the build too.

Rule 3 · Shared means generic

A component enters `packages/ui` only when both products need it and it carries no business meaning. The supplier grid and the 3D viewport stay in their apps.

Rule 4 · The domain has no UI imports

In Surface 2, `domain/` (commands, snapping, rules, BOM) imports neither React nor three.js, so it is unit-tested in isolation and can move into a web worker or onto the server unchanged.

Rule 5 · Features never import each other

They share through the URL, the query cache or the store. The AI panel reads the grid's selection from the store; it never imports the grid.

Rule 6 · Enforced and owned

Nx tags (`type:app|feature|data|domain|ui` , `scope:procurement|configurator|shared`) fail CI on a wrong import. CODEOWNERS per package; the design-system team approves `ui` and `tokens`.

Layer	May import	Surface 1 · `apps/procurement`	Surface 2 · `apps/configurator`
`app/`	All layers below	Shell, router, providers, runtime config, error boundaries	Host bridge, theme adapter, error boundary with a WebGL fallback
`features/`	State, data, scene, domain, packages	One folder per screen family	Panels: palette, items, properties, rule check, BOM, assistant
`domain/`	Nothing (pure TypeScript)	(none: rules live on the server)	Layout model, commands and history, snapping, clearance rule, BOM
`state/` / `data/`	Domain, `api-types`	Query keys, API client, realtime client, drafts, uploads	Layout store, API client, realtime client, proposal state
`scene/`	State, domain	(none)	Floor, instanced items, overlays, asset loader, camera

In Part B: the code follows these rules: `domain/` imports only itself, `scene/` and `ui/` never import each other, and all 47 tests run in Node without a browser. A lint rule to enforce them is the next step.

3.5 Why this structure, and why not micro-frontends

The brief asks for two products that **share a design system** but **release on their own schedules**. That is a code-sharing problem, not a problem of many teams composing one page. So we share code **at build time** through versioned packages, and each surface ships as one tested app. Micro-frontends (Module Federation) share code **at run time** instead; here that costs more than it gives.

Concern	Our structure: monorepo + versioned packages	Micro-frontends: Module Federation
Independent releases	Yes: each app pins package versions and opts in (Section 3.2)	Yes, but a remote's release changes a live app that never tested it
Safety	What CI tested is exactly what ships	Version skew at run time: host and remote can disagree on React, the design system or a contract, and it shows up in production
Load time on the VPN	One optimised bundle per route; shared code tree-shaken at build time	Extra manifest and remote-entry requests (about 150 ms each on the VPN); shared libraries are de-duplicated only when versions match
The 3D editor	Its own app in an iframe: own memory, own crashes (Section 7.1)	Runs in the host's page: up to 1 GB and WebGL failures shared with the host's app
Team fit	One frontend team; ownership by folder, tags and CODEOWNERS	Pays off when many teams own parts of one page and must deploy without coordinating; not the case here
Types and debugging	One type check across packages; one build and one set of source maps per app	Types across remotes need extra tooling; one error can span several deployments

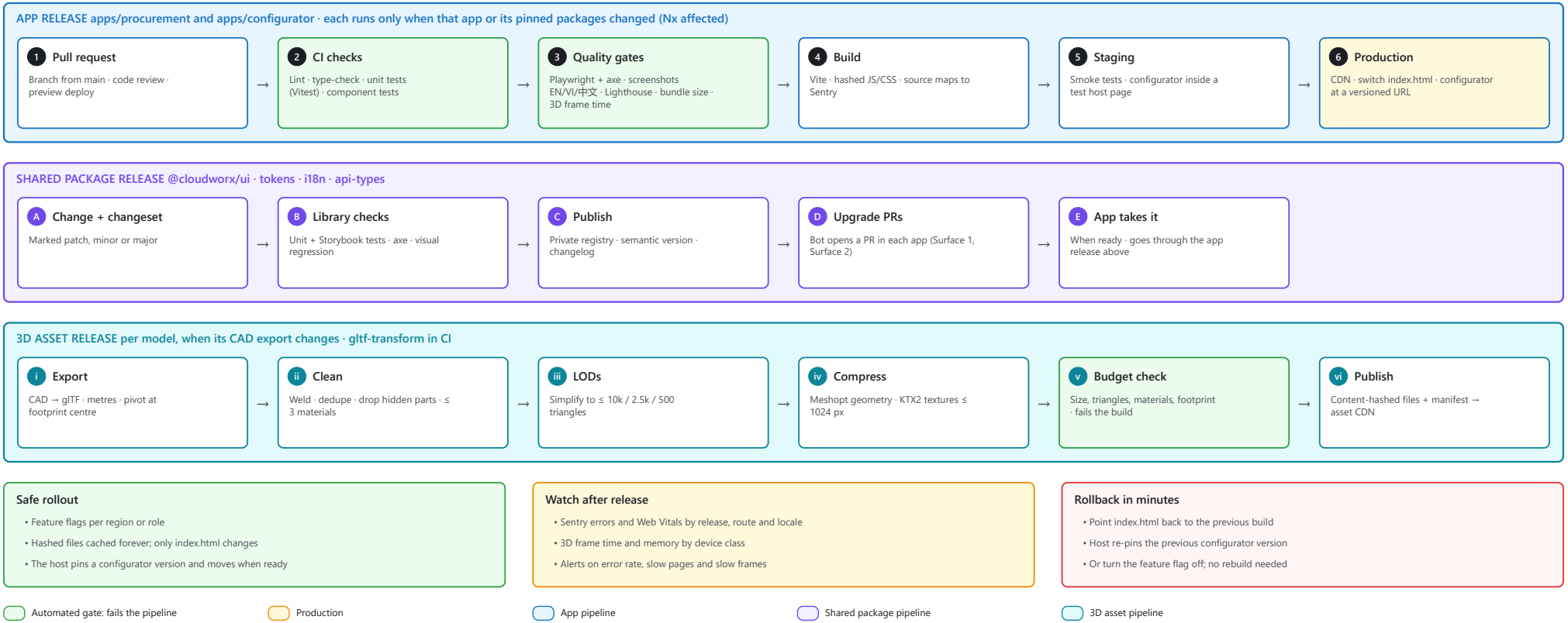
The trade-offs we accept: a shared change needs a package release and an opt-in per app (minutes with changesets); one repository needs good CI caching (Nx affected) to stay fast. **When we would revisit:** if several independent teams later own separate areas of the procurement workspace. Module Federation stays only as the **embedding fallback** for hosts that forbid iframes (Section 7.1), where the typed host contract limits the coupling.

Also rejected: separate repositories (versions drift; cross-cutting changes need several pull requests); one app for both surfaces (one release train, and the host would load procurement code); copying components into each app (copies drift; every fix made twice).

Build and release

Build & release · frontend

Every change goes through the same automated gates. Each app deploys on its own; shared packages are versions that apps take when ready; 3D models have their own pipeline.



4 State and data

Server state and caching, optimistic updates, autosave and drafts on long forms, live updates over WebSocket, behaviour on a flaky VPN, the role-based home and dashboards.

4.1 Each kind of state has one home

Kind	Examples	Lives in	Why
Server state	Tasks, suppliers, RFQs, quotes, approvals	TanStack Query	Caching, deduplication, retries, offline pause and optimistic rollback built in
Grid rows	Blocks of the 50,000 / 200,000-row grids	AG Grid SSRM cache	Block cache designed for very large server-side datasets (Section 5)
Layout document	Placed items, history, selection, AI proposal	Zustand store (Surface 2)	Edited at 60 fps; the 3D view and panels must share one copy (Section 8)
URL state	Filters, sort, view, selected task, wizard step	TanStack Router search params	Bookmarkable, shareable, survives reloads; typed
UI state	Open panels, row selection, density	Zustand	Small, no provider, selective re-renders
Drafts	Unsaved form input	IndexedDB + server drafts	Survives tab crashes, VPN drops and device changes

4.2 Server state and caching

- **Query keys mirror the API:** `['tasks', {status}]`, `['rfq', id]`, `['rfq', id, 'quotes']`. Invalidating a prefix refreshes everything beneath it.
- **Freshness per kind of data:** reference data is fresh for 24 h, lists for 30 s; the open record revalidates on focus and reconnect.
- **Screen-shaped endpoints:** the BFF serves the home page in one call, saving three VPN round trips on the most important screen.
- **Bounded memory:** unused queries are collected after 5 minutes; the persisted cache (inbox and home only) is cleared at sign-out and never used on shared VDI profiles.
- **Several tabs:** sign-out, language and draft changes reach every open tab through `BroadcastChannel`; each tab keeps its own WebSocket and cache.

4.3 Optimistic updates, done safely

```
Approve task  →  remove it from the cached list immediately          (feels instant)
              →  POST /tasks/{id}/approve   Idempotency-Key: 7f3c…   If-Match: "v4"
   200        →  invalidate ['tasks'] and ['rfq', id]                 (resync counts)
   412/409    →  restore the task, show "Already approved by Wei L." (no silent double action)
   network    →  restore the task, mark it "Not sent · Retry"
```

- **Idempotency keys** make retries safe; **ETags with** `If-Match` catch concurrent edits.
- Optimism is used where rollback is cheap and visible (approvals, task actions, comments, 3D edits). Irreversible commands (award, submit onboarding) wait for the server.

4.4 Autosave and drafts on long forms

When	What happens
Every change	The draft is written to IndexedDB at once
1.5 s after typing stops, on field blur, on step change	`PATCH /drafts/{id}` with the draft version; the header shows "Draft saved · 10:42"
Offline	Local writes continue; "Saved on this device · will sync"; the server save retries on reconnect
Reopening	The newer of local and server versions wins; if both changed, the user chooses with a field-level comparison
Resume	An inbox task deep-links to `/onboarding/{draftId}/step/3`; server drafts are kept 30 days

4.5 Live updates over WebSocket

- **One connection per tab**, opened after the shell renders. Screens subscribe to topics (`tasks:me`, `rfq:2041`, `layout:er-104`) and unsubscribe when they unmount.
- **Every event has a sequence number.** On reconnect the client sends its last `seq` and the server replays a 15-minute buffer; for a larger gap it sends `resync` and the client invalidates the active screen.
- **Events update the cache, never components directly;** handlers are idempotent by `seq`. New inbox items never shift the list under the cursor: a "1 new task" pill lets the user choose.
- **Heartbeat** every 25 s; reconnect with exponential backoff (1 s → 30 s) plus jitter, so thousands of clients do not reconnect at once after a VPN gateway restart.

4.6 Behaviour on a flaky VPN

State	Detected by	What the user sees and what the app does
Degraded	Requests over 3 s, heartbeat late	"Slow connection" chip; skeletons; non-urgent prefetching and 3D LOD refinement paused
Offline	Heartbeat lost, failed requests	Banner "You're offline. Changes are saved on this device." Cached screens stay readable; drafts, comments and 3D edits queue in IndexedDB, so closing the tab loses nothing; **approvals and awards are disabled**
Reconnected	Heartbeat resumes	Queued writes are sent in order with their idempotency keys; the WebSocket resumes from the last `seq`; the active screen refetches

4.7 Role-based home and permissions

Buyers, suppliers and approvers each start on their own home page and see only the pages they can use. One build serves every role; the role decides what is shown, and the server decides what is allowed.

Role (wireframe)	Home shows	Menu
Buyer (01)	My tasks, KPI tiles, expiring certificates, live activity, RFQs in progress	Home, Inbox, Suppliers, Documents, RFQs, Approvals, Dashboards
Supplier (04, 08)	Onboarding progress, documents to upload, quote invitations	Home, Inbox, My company, Documents, RFQs & quotes
Finance approver (06)	Approvals waiting, budget use	Home, Inbox, Approvals, Reports

- **Where roles come from:** the BFF session (company OIDC sign-in) returns the user's roles and permissions in `GET /me` at start-up; a role change arrives as a WebSocket event and the shell reloads them.
- **The home is composed per role:** the server returns the role's widget list, and `GET /home` brings the first data for every widget in one call (Section 4.2), so it is the 2.5 s "first useful screen" (Section 11.1). Each widget is a lazy module with its own query, skeleton and error boundary, so one slow widget never blocks the page.
- **Menus and routes follow permissions:** every route and menu item declares the permission it needs; the shell hides the rest, and a direct link to a forbidden page shows "You don't have access" instead of loading it. A user with two roles (a buyer who also approves) gets both sets.
- **The server is the authority:** hiding is for clarity only. Every API call is checked on the server, and fields a role may not see never reach the browser (Section 5.1).

4.8 Dashboards

- **Totals come from the server:** each widget calls `GET /dashboards/{id}/widgets/{widget}` with the dashboard filters from the URL; the browser never loads 50,000 rows to draw a chart.
- **Charts:** Apache ECharts (canvas, fast with many points), loaded only on dashboard routes (D14) and themed from our tokens; axis dates, numbers and currencies use `Intl` for the viewer's locale.
- **Freshness:** widgets refresh on a `metrics.updated` WebSocket event, or every 5 minutes, one request per widget.
- **Drill-down:** clicking a bar or a KPI tile opens the grid with that filter (wireframe 01).
- **Accessible:** each chart has a title, a one-sentence summary and a "View as table" twin with the same data; series are told apart by labels and patterns, not colour alone.

5 Grids and forms

Virtualised grids with saved views, bulk actions and server-side filtering; multi-step forms with validation, autosave and resume.

5.1 Large grids: suppliers (50,000) and documents (200,000)

AG Grid Enterprise, server-side row model, themed with our CSS variables. The server leaves out fields a role may not see (e.g. bank details).

```
POST /grids/suppliers/rows
{ startRow: 200, endRow: 300, sort: [{ col: "certExpiry", dir: "asc" }],
  filter: { country: ["IN"], status: ["ACTIVE"] }, search: "brake", viewId: "v-81" }
→ { rows: [ …100 rows… ], rowCount: 1284 }
```

Concern	Approach
Memory	100-row blocks, at most 20 cached (≤ 2,000 rows); about 30 DOM rows at any time; column virtualisation for wide views
Filter, sort, search	On the server, with locale-aware collation and accent-insensitive search ("Ha Noi" finds "Hà Nội"); the query lives in the URL
Saved views	Server-stored columns, sort, filters and density; personal or shared; a default per role
Bulk actions	Selection is a list of IDs or **a filter with exclusions**; the server runs a job (`202 + jobId`) and reports progress on the WebSocket
Export	A server-side job produces the file; the browser never assembles 200,000 rows
Accessibility	ARIA grid roles and full keyboard navigation; sort/filter changes and counts announced; status as text, not colour alone

RFQ comparison (40 × 8): client-side rows; supplier columns generated from the data (any number), pinned line-item column and header, lowest price per line marked with an arrow and text as well as colour, amounts converted with the original and rate date on hover.

5.2 Long multi-step forms: supplier onboarding

Concern	Approach
Structure	The wizard is configuration: each step has fields and a **Zod schema**; country rules (e.g. the Vietnamese tax code length) come from the server
Performance	**React Hook Form** keeps inputs uncontrolled, so a 60-field step does not re-render per keystroke
Validation	On blur, then on change once in error; the whole step on "Next"; the server validates again and returns error codes mapped to fields
Autosave and resume	As Section 4.4
Uploads	Chunked, resumable uploads to pre-signed URLs; virus-scan status arrives as an event
Accessibility	Labels on every field, errors linked with `aria-describedby` , an error summary with links, focus to the first error, step changes announced

6 Localisation and accessibility

One build in three languages, text expansion, CJK line breaking and locale formats; keyboard and screen-reader paths for the main flows, including the 3D view.

6.1 One build, three languages

Part	Approach
Choosing the language	Surface 1: saved profile → browser → English. Surface 2: the **host's locale** from the context hand-off (Section 7.2), so the editor always matches the host page
Translations	JSON per language and namespace, ICU message format, hashed on the CDN; only the active language and open screen are downloaded (5–20 KB each)
Switching	At runtime with no reload: load files, re-render, set `<html lang>`, switch formatters (Part B does this for EN/VI)
Safety	Typed keys fail the build when missing; hard-coded strings are a lint error; a missing translation falls back to English and is reported
Server messages	The API returns error codes, not text; emails and notifications are rendered server-side in the user's language
Long-form content	Help, country document requirements and legal text come from a headless CMS per locale; UI strings stay in the repository

6.2 Text expansion, CJK and locale formats

Text expansion

Vietnamese runs about 30–40% longer than English. No fixed widths; buttons and tabs grow, labels wrap, grid cells truncate with a tooltip. A pseudo-locale in CI screenshots catches overflow and untranslated text.

CJK line breaking

Under `:lang(zh-CN)`: `line-break: strict` (no line starts with 。or ，), `word-break: normal`, `overflow-wrap: anywhere` for long part codes, no fake italics. Noto Sans SC loads only for Chinese, subset, with `font-display: swap`.

Typing with an input method

Chinese pinyin and Vietnamese tone-mark input compose text before it is committed. Search-as-you-type, shortcuts and Enter ignore keys while `isComposing` and act on `compositionend`, so half-typed text is never searched or submitted.

Typing numbers

Number and currency fields parse input in the user's locale ("1.840.000,50" in Vietnamese means 1,840,000.50) and also accept plain digits; the formatted value shows on blur, so a separator can never change an amount.

Same value	English (India)	Vietnamese	Chinese
Amount, INR quote	₹18,40,000.00	1.840.000,00 ₹	₹1,840,000.00
Amount, VND quote	₫1,25,00,00,000	1.250.000.000 ₫	₫1,250,000,000
Amount, CNY quote	CN¥1,56,400.00	156.400,00 CN¥	¥156,400.00
Amount, USD quote	$22,150.00	22.150,00 US$	US$22,150.00
Date	17 Oct 2026	17 thg 10, 2026	2026年10月17日

Distance in the 3D editor	0.5 m	0,5 m	0.5 m

Values are the browser's actual `Intl` output. **The currency comes from the data and only the style from the locale.** Times are stored in UTC and shown in the viewer's time zone; RFQ deadlines show the zone explicitly.

6.3 Accessibility approach (WCAG 2.2 AA)

- **Accessible by construction:** React Aria for interactive components; semantic HTML elsewhere.
- **Visible focus**, a skip link, logical tab order, focus moved into dialogs and back, targets at least 24 × 24 px (44 px on mobile).
- **One announcer** (a polite live region) for asynchronous events: "Draft saved", "New task", "Undone: move D-01".
- **Never colour alone:** statuses carry text or an icon; violating 3D items are red *and* listed with a warning icon and a message.
- **The 3D view has a text twin:** the placed-items list, properties and rule-check panels expose the whole scene to a screen reader.
- **Motion, contrast and zoom:** `prefers-reduced-motion` turns off animations (Part B does this) and makes camera moves jump; `forced-colors` keeps borders and focus visible; every screen works at 200% zoom and reflows at 320 px.
- **Tested continuously:** axe fails the build; keyboard paths below are scripted in Playwright; NVDA + Edge and TalkBack checks before each release.

6.4 Keyboard and screen-reader paths for the main flows

Flow 1 · Approve a task from the inbox (buyer or approver)

Keys	Screen reader says
Open inbox	"Inbox, 12 open tasks, 2 due today, 1 overdue."
`Tab` into the list, `↓` / `↑`	"Approve RFQ-2039, gaskets and seals. Approval, 1 day overdue. 1 of 12."
`Enter`	Detail opens, focus on its heading: "Recommended supplier Delta Seals, 6 lakh 72 thousand rupees."
`Tab` to Approve, `Enter`	"Approved. Next task: Answer supplier question, RFQ-2040." Focus moves to the next task.

Flow 2 · Find suppliers and request documents in bulk (buyer)

Keys	Screen reader says
View menu, arrows, `Enter`	"View: Active suppliers, India. 1,284 suppliers."
Arrows between grid cells	"Bharat Brakes Limited. Certificate expiry, 5 November 2026. Row 1 of 1,284."
`Space`, or "Select all matching"	"All 1,284 matching selected."
Request documents, confirm	"Request sent. Job running." Later: "Job finished. 1,281 sent, 3 failed."

Flow 3 · Complete an onboarding step with an error (supplier)

Keys	Screen reader says
Open from the inbox task	"Supplier onboarding, step 3 of 6, Bank details. Draft saved 10:42."

`Tab` through fields	"Tax code, required." On leaving: "Invalid: tax code must be 10 or 13 digits."
`Enter` on Next with errors	Focus to the error summary: "1 error", with a link to the field.

Flow 4 · Compare quotes and award (buyer)

Keys	Screen reader says
Arrows in the matrix	"Front brake pad set, Bharat Brakes, 396 rupees, lowest on this line."
Award, choose supplier, add reason, confirm	"Award sent for approval to Wei L. RFQ locked."

Flow 5 · Place, move and fix an item in the 3D editor (sales engineer)

Keys	Screen reader says
`Tab` to *Placed items*, `↓` / `↑`, `Home` , `End`	"C-03, control cabinet, 4, 2.5 metres. Closer than the minimum clearance to another item." The item is selected in 3D too.
Arrow keys (one 0.5 m cell; Shift: 1 m)	"Done: move C-03. Now at 4.5, 2.5 metres. Too close to D-07 (0.4 metres)."
`R` / `Delete` / `Ctrl+Z`	"Done: rotate C-03." / "Done: delete C-03." / "Undone: delete C-03."
`Tab` to *Rule check*, `Enter` on a message	"C-03 and D-07 are 0.4 metres apart." Selects the item.

All of Flow 5 is implemented in Part B, including the arrow-key nudge and the announcements quoted above.

7 Embedding in the host

Iframe or micro-frontend and why, the `postMessage` contract (context hand-off, token refresh, resize, navigation, errors), origin checks, CSP and cookie constraints, and matching the host design language.

7.1 Iframe, with a micro-frontend fallback

	Iframe (chosen)	Micro-frontend
Isolation	Separate JavaScript, CSS and memory	Shares the host page; CSS and globals can clash
If the 3D view fails	Only the frame is affected; the host keeps working	A WebGL crash or a 1 GB heap can freeze the host's sales app
Releases and React version	Fully independent	Coupled to the host's runtime and shared dependencies
Security	A hard wall between two companies' code and data	Both share everything in the page
Cost	A message contract, resizing, cookie limits	Direct props and events

Why iframe: the editor is the heaviest thing on the page (up to 1 GB, WebGL), it belongs to a different company, and it must ship on its own schedule. **Fallback** if a host forbids iframes: the same app as a web component in Shadow DOM, loaded with Module Federation, speaking the **same contract** over DOM `CustomEvent`s. Only the transport changes, inside `packages/host-bridge`.

7.2 The postMessage contract

```
type Message = {
  protocol: 'cwx-embed'; version: 1;      // anything else is ignored; version allows evolution
  id: string; replyTo?: string;            // request / reply pairing
  type: string;                            // e.g. 'host:init'
  payload: unknown;                        // validated against a zod schema per type
};
```

Direction	Message	Purpose
App → host	`app:ready`	"Loaded; send context." Sent once the shell renders
Host → app	`host:init`	**Context hand-off:** user (id, name, roles), project and layout ids, locale, time zone, theme tokens, feature flags, access token and expiry, optional deep link
App → host	`app:token-request`	**Token refresh** two minutes before expiry, or after a 401
Host → app	`host:token`	New token and expiry, as a reply to the request
Host → app	`host:theme`, `host:locale`	The host changed theme or language; applied live
App → host	`app:resize`	**Resize:** preferred height when the host lets content size the frame
App → host	`app:navigate`	**Navigation** outside the editor, e.g. "open product datasheet"; the host owns the URL
Host → app	`host:navigate`	Open a layout or select an item (deep link)

App → host	`app:dirty`	Unsaved changes exist, so the host can confirm before leaving
App → host	`app:error`	**Errors** as codes with a `recoverable` flag (Section 7.4)

```
Host page                                   Configurator (iframe)
  | creates <iframe src="https://configurator…/v2.3/">  | renders shell + floor skeleton
  |◄──────────────────── app:ready ──────────────────────|
  |──────────────────── host:init ──────────────────────►| validates, sets locale + theme,
  |   { user, project, locale, theme, token, exp }      | loads layout (REST ⑩), opens WSS ⑪
  |                        …                            |
  |◄──────────── app:token-request (exp − 2 min) ────────| pauses API calls if a 401 arrives
  |──────────── host:token { token, exp } ──────────────►| retries the paused calls
 No host:init within 10 s → "Couldn't connect to the host. Retry", and app:error HOST_TIMEOUT.
```

7.3 Token handling

- The host owns sign-in (OIDC). The editor never sees a password and never runs its own login.
- The access token is **short-lived (≈ 15 min), audience-scoped to the Configurator API**, kept **in memory only**: never in the URL, storage or logs.
- On a 401 the API client pauses its queue, requests a token, and replays; a second failure sends `app:error` `AUTH_FAILED` so the host can re-authenticate. Only **one refresh runs at a time**; other failed calls wait for it.
- **The WebSocket:** browsers cannot add an `Authorization` header to it, and the token never goes in a URL. So `POST /ws-ticket` (with the bearer token) returns a **single-use ticket valid for 30 s** that opens the socket. After each refresh the new token is sent over the open socket; otherwise the server closes it when the token expires.
- The Configurator API validates issuer, audience, expiry and signature on every call; the browser is never trusted for permissions.

7.4 Resize, navigation and errors

Concern	Approach
Resize	Default: the host gives the frame a full panel and the editor fills it, adapting its layout. Optional: a `ResizeObserver` reports content height (throttled to one message per frame). Dialogs and menus are designed to stay inside the frame; full screen uses `allow="fullscreen"`.
Navigation	Inside the editor: an in-memory router, not the host's URL. Leaving the editor: `app:navigate`, and the host decides. Deep links arrive in `host:init` or `host:navigate`.
Keyboard focus	`Tab` moves in and out of the frame in page order, never trapped; the frame has a title
Errors	`HOST_TIMEOUT`, `AUTH_FAILED`, `LAYOUT_NOT_FOUND`, `WEBGL_UNAVAILABLE` (the editor falls back to its list and BOM view), `ASSET_LOAD_FAILED` (retry with backoff). Each side shows the code in its own language and style.

7.5 Origin checks

```
const ALLOWED_HOSTS = runtimeConfig.allowedHostOrigins;     // exact origins, per environment

addEventListener('message', (e) ⇒ {
  if (!ALLOWED_HOSTS.includes(e.origin)) return;            // 1. the right website?
  if (e.source ≠= window.parent) return;                    // 2. from our actual parent?
  const msg = MessageSchema.safeParse(e.data);              // 3. the right shape?
  if (!msg.success || msg.data.protocol ≠= 'cwx-embed') return;
  handle(msg.data);
});
window.parent.postMessage(message, hostOrigin);             // never '*'; protects the token
```

The host applies the same three checks to messages from the iframe. Origins are compared exactly (not "contains hostcorp"), against a runtime allow-list per environment.

7.6 CSP and cookie constraints

```
Content-Security-Policy:
  frame-ancestors https://select.hostcorp.com;          approved hosts only (Section 7.5)
  default-src 'self';
  script-src 'self' 'wasm-unsafe-eval';                 Meshopt and KTX2 decoders (WASM)
  worker-src 'self' blob:;                              decoders run in web workers
  connect-src 'self' https://api.configurator… wss://live.configurator…
https://*.ingest.sentry.io;
  img-src 'self' blob: data: https://assets.configurator…;
  font-src 'self' https://fonts.hostcorp.com;           the host's brand font
  style-src 'self';
```

- **On the host:** `frame-src https://configurator…`, and the iframe with `sandbox="allow-scripts allow-same-origin allow-forms allow-downloads"`, `allow="fullscreen; clipboard-write"` and a `title` for screen readers.
- **Cookies:** in another company's page ours are third-party and Chrome/Edge restrict them, so we **use no cookies for auth**, only the host's bearer token. If one is ever needed: `SameSite=None; Secure; Partitioned` (CHIPS), or a host subdomain.
- **Storage is partitioned** per host too; the Cache API and IndexedDB still cache models, separately per host.

7.7 Matching the host design language

```
host:init → theme: { "color-primary": "#c8102e", "font-family": "Host Sans",
                     "radius": "2px", "density": "compact", "mode": "light" }
```

1. The theme adapter **validates** each value (colours, lengths, allowed fonts), so a host cannot inject CSS.
2. It maps them onto our **brand and semantic tokens** (Section 3.3); every component re-styles at once.
3. It checks **contrast**: a host colour that fails WCAG AA gets the nearest passing shade.
4. Missing values fall back to our defaults; `host:theme` applies changes live (e.g. dark mode).

The host controls colour, font, radius, density and mode. We keep the editor's layout, the 3D visuals and accessibility.

8 The 3D editing workspace

Scene structure, selection and transform editing, snapping, undo and redo, showing rule violations, keeping the 3D view in sync with the list and property panels, and wiring. This section describes what Part B implements, then how it scales to 2,000 items.

8.1 Scene structure

```
<Canvas>                                  React Three Fiber, rendered on demand*
├─ Static       floor plane · 0.5 m grid · walls                    a handful of draw calls
├─ Items        one InstancedMesh per model × material × LOD         per-instance matrix +
status colour
├─ Overlays     selection outline · 0.6 m clearance zone · violation links* · AI ghost layer*
│               (never raycast, so clicks always reach the floor and items)
├─ Labels       HTML label for the selected item only
└─ Camera       orbit / pan / zoom, fitted to the room on resize
```

* Design only, not in Part B; Part B renders every frame so its fps readout keeps measuring. Part B builds the rest with one InstancedMesh per part per item type, without LODs.

The domain works in metres from the room's north-west corner; the scene converts to centred world coordinates. The layout is the single source of truth: **the scene is a pure function of the store**, so nothing in three.js has to be kept in sync by hand.

8.2 Selection and transform editing

- **Picking:** instanced meshes report `instanceId`. At 2,000 items picking uses the same spatial hash as the rule check: the floor point under the pointer finds the item whose footprint contains it in O(1), instead of ray-testing every triangle.
- **Move:** items move on the floor plane (equipment does not float), with pointer capture so a fast drag never "drops" the item. The drag preview lives in local state; **one command is committed on release**.
- **Rotate:** 90° steps (`R`), re-snapped so the turned footprint stays on the grid and on the floor.
- **Properties:** position (as the north-west corner, always a clean grid value), rotation and label; edits commit on Enter or blur, one undo step each.
- **Multi-select, copy and paste:** the selection is a list with a primary item (Ctrl/Shift-click, list ranges, Ctrl+A). Group move, rotate, delete, cut and paste are each **one batch command**; moves are clamped for the group as a whole, so a group stops at a wall instead of being squashed against it. Pasted copies get the next labels and become the selection.
- **Keyboard:** list navigation, `R` , `Delete` , `Ctrl+Z/Y` ; arrow keys nudge one cell (Section 6.4).

8.3 Snapping to the grid

The north-west corner of the rotated footprint snaps to the nearest 0.5 m line and is clamped so the whole footprint stays on the 20 × 12 m floor. A quarter turn swaps width and depth before snapping. Floating-point noise is rounded at millimetre precision. Unit-tested in Part B.

8.4 Undo and redo: every edit is a command

```
type Command = { kind: 'add' | 'remove' | 'move' | 'rotate' | 'update' | 'batch';
                 target: string;                    // "C-03", for messages and announcements
                 apply(layout): Layout; revert(layout): Layout };
```

- **Exact restore:** commands store previous values, so undo returns the exact position and even the item's place in the list.
- **Batches:** loading a layout or accepting an AI proposal is one command, so one `Ctrl+Z` reverts it.
- **No-ops are dropped:** a drag that ends where it started records nothing. History is capped at 200 steps.
- **Same commands to the server:** each committed command is sent as an operation with the layout version (`POST /layouts/{id}/ops`, `If-Match`). The server re-runs the rules authoritatively; on a version conflict, operations on different items rebase, otherwise the layout reloads with a message.

8.5 Showing rule violations

- **Rule:** any two items closer than 0.6 m edge to edge (footprint to footprint), overlaps included.
- **Speed:** a spatial hash compares each item only with items in neighbouring cells. **Measured: 4.8 ms for 2,000 items in a realistic layout, against 342 ms for all pairs**, so it runs on every drag move. Denser future rules (door swing, cable length) move to a web worker; the domain has no UI imports, so that is a move, not a rewrite.
- **Live feedback:** the check runs on the drag preview and the placement ghost, so a conflict shows *before* the user commits.
- **Display:** red items in 3D (per-instance colour), a warning icon in the list, and a message naming both items in the rule panel ("C-03 and D-07 are 0.4 m apart"); clicking it selects the item. The live region announces new conflicts.

8.6 Keeping the 3D view, list and properties in sync

One Zustand store holds the layout, the selection and any AI proposal. The 3D view, the placed-items list, the properties panel and the rule panel are all **readers of the same store**, and selection is a store field, so selecting in the list highlights in 3D and the list scrolls to an item picked in 3D. Derived data (violations, BOM) is computed once per layout change and shared. At 2,000 rows the list is virtualised (TanStack Virtual); Part B's 500 rows use `content-visibility`.

8.7 Wiring

Engineers connect equipment with cables, e.g. a drive's motor output to a cabinet. Design only; not in Part B.

Concern	Approach
Model	Each catalogue model declares its **ports** (position on the footprint, power or control, rating). A wire is `{ id, from: { itemId, port }, to: { itemId, port }, cableType }`, stored in the same layout document as the items.
Editing	Click a port, then a compatible one (`Esc` cancels), or use "Connect to…" in the properties panel, which also lists each item's wires for screen readers. Wire edits are **commands**, so undo and redo work as for any edit (Section 8.4).
Routing	Orthogonal routes along the grid, like cable trays. Moving an item re-routes its wires live and commits once, with the move; deleting an item removes its wires in the same batch.
Rules	In the same rule panel as the clearance check: a required port left unconnected, incompatible ports, and a cable longer than its type allows ("D-03 → C-01: 34 m, limit 30 m").
Bill of materials	Cable metres per type, counted from the scene like the equipment.
Drawing	All wires in one line or instanced-tube mesh, rebuilt only when wires change: one or two draw calls, whatever their number.

9 3D asset optimisation

From a raw 40 MB CAD export to a web-ready model: simplification and levels of detail, Meshopt compression, KTX2 textures, instancing, culling, lazy loading and caching, with target numbers per model and per scene.

9.1 The pipeline (CI, per model)

#	Step	What happens (gltf-transform / meshoptimizer)
i	Export	CAD → glTF in metres, Y-up, pivot at the footprint centre on the floor; units and bounding box checked against the catalogue footprint (±1 cm)
ii	Clean	`dedup` , `weld` , `prune` ; drop internal parts invisible from outside (bolts, PCBs inside a closed cabinet); merge meshes by material; ≤ 3 materials per model
iii	LODs	`simplify` to three triangle budgets (below); LOD2 is close to a shaped box with baked colour
iv	Compress	`quantize` + `meshopt` (EXT_meshopt_compression) for geometry; textures resized to ≤ 1024 px and encoded to **KTX2** (ETC1S for colour, UASTC for normal maps)
v	Budget check	glTF validator + a budget script (size, triangles, materials, footprint); **any breach fails the build**
vi	Publish	Content-hashed files and a manifest (model → LOD URLs, sizes, footprint) on the asset CDN, immutable

9.2 Targets per model

	Raw CAD export	LOD0 · near (< 6 m)	LOD1 · mid (6–15 m)	LOD2 · far / first load
Triangles	200,000 – 2,000,000	≤ 10,000	≤ 2,500	≤ 500
Geometry (Meshopt + gzip)	2 – 40 MB	≤ 250 KB	≤ 70 KB	≤ 20 KB
Materials (draw calls per bucket)	10 – 60	≤ 3	≤ 2	1
Textures	many PNG/JPG, up to 4K	shared per model · KTX2 · ≤ 1024 px · ≤ 300 KB		none (vertex colour)
Whole model	**2 – 40 MB**	**≤ 650 KB for all three LODs and textures: at least 95% smaller**		

9.3 Targets per scene (2,000 items)

Metric	Target	How
Draw calls	≤ 260 (budget 300)	≤ 40 distinct models × ≤ 3 materials × ≤ 2 LODs visible at once ≈ 240, plus ~20 for floor, grid and overlays. They scale with distinct models, not items
Triangles in view	≤ 2 million	Near items at LOD0, most at LOD1/LOD2; frustum culling
First 3D view	< 1 s on 10 Mbps	LOD2 of every model in the room first: 40 × 20 KB ≈ 0.8 MB
Full detail	≤ 26 MB, streamed	LOD1/LOD0 by distance, idle time and network state; nothing blocks editing

Repeat visit	≈ 0 MB	Immutable CDN URLs + Cache API
GPU memory	≤ 150 MB	KTX2 stays compressed on the GPU (about 4–6× smaller than PNG)
Tab memory	≤ 600 MB (budget 1 GB)	Shared geometry per model; LRU eviction of unused LOD0 when near the budget

9.4 At runtime

- **Instancing:** one `InstancedMesh` per model × material × LOD. Each frame the camera moved, instances are re-bucketed by distance (throttled), and per-instance frustum tests skip off-screen items: 2,000 sphere tests cost well under a millisecond. Part B proves the principle: **9 draw calls for all items whatever their number, 15 in total at 500 items**.
- **Decoding off the main thread:** the Meshopt decoder and the KTX2 (Basis) transcoder are WebAssembly in web workers, so loading never stutters the editor (CSP allows `'wasm-unsafe-eval'`).
- **Lazy loading:** the manifest loads with the layout; a model downloads on first use; placeholders (the footprint box, as in Part B) show instantly.
- **Caching:** content-hashed URLs, long immutable CDN caching, and the Cache API in the browser, so the second visit is close to zero bytes.

Rejected

- **Draco** instead of Meshopt: comparable compression, but slower decoding on weak CPUs, a larger decoder, and it does not compress animation or instance data. Meshopt also pairs with quantisation and is three.js's default path.
- **PNG/JPEG textures:** they expand to full size in GPU memory; KTX2 stays compressed.
- **Simplifying at runtime:** CPU cost on every load on the weakest machines; done once in CI instead.
- **One mesh per item:** 2,000+ draw calls. **Merging the room into one static mesh:** few draw calls, but every edit would rebuild it.

10 AI assistant panel

Streaming replies, previewing the agent's proposed change in the 3D view before the user accepts it, and undoing an accepted change. The mechanism is shared by both surfaces.

10.1 How a reply streams

```
1  POST /assistant/messages  { conversationId, text, context }      REST    → 202 { runId }
2  WS   assistant.delta  { runId, text: "C-03 is 0.4 m from D-07…" } WSS     (many, in order by
seq)
3  WS   assistant.done   { runId, sources, proposal? }
   Stop → POST /assistant/runs/{runId}/cancel                          REST
```

- **Only REST and the existing WebSocket**, the two channels the brief fixes; reconnect and replay (Section 4.5) also protect a reply in progress.
- Tokens render as they arrive; the finished reply, not each token, goes to the live region.
- The client sends **references** (screen, layout id and version, selection, violation ids), never data; the server resolves them with the user's permissions and alone calls the LLM. Content is data, not instructions.
- **Limits and failures:** per-user rate and time limits; a broken stream keeps its partial reply ("Incomplete · Retry"); proposals are logged for audit.

10.2 A proposal is a set of commands

```
proposal: { layoutVersion: 41,
            summary: "Move C-03 0.5 m east to clear D-07",
            changes: [ { op: "move", itemId: "c03", x: 4.9, z: 2.3 } ],
            resolves: [ "c03|d07" ] }
```

The agent never edits the layout. It returns the same operations the editor already understands (Section 8.4), so the client can validate them with its own rules and the user stays in control. In Surface 1 the same pattern previews proposed actions (for example a document request to 2 suppliers) and confirms them through the normal REST command with the user's own permissions.

10.3 Preview, accept, undo

Step	Behaviour
Validate	The client checks the items exist, snaps positions, applies the changes to a *copy* of the layout and runs the clearance rule on it: "resolves 1 issue · creates 0". New violations are shown, not hidden
Preview	A **ghost layer** draws moved items translucent at their new place, with a dashed arrow from the old position; originals are dimmed. Nothing in the real layout changes. The same change is listed in text for screen-reader users
Accept	The changes apply as **one batch command** ("Assistant: move C-03"), go to the server as operations with the version, and announce "Done: assistant fix"
Stale	If the layout version moved on since the proposal, it is re-validated; if it no longer applies cleanly it is marked stale with "Ask again"
Reject / undo	Reject removes the ghost layer. After accepting, one `Ctrl+Z` reverts the whole fix like any other edit; the server keeps the operation log for audit

11 Performance and quality

How the budgets are met, how they are measured in CI and in the field, error reporting, and the testing strategy.

11.1 The 2.5 s first useful screen (Surface 1)

Assumptions: 10 Mbps, 150 ms round trips over VPN, a mid-range 4-core CPU. "First useful screen": the role's home with its task list interactive.

Step	Budget	How
DNS + TCP + TLS 1.3	0.45 s	3 round trips; one origin for app and BFF
HTML	0.15 s	2 KB shell; preloads critical files; **starts the home-data request inline** (0.4 s, overlapping the steps below)
Critical resources	0.35 s	≤ 170 KB JS + 25 KB CSS + 10 KB locale + 30 KB font subset, compressed
Parse and run JS	0.35 s	Route-split bundles; grids, charts and the assistant load when used
Render	0.10 s	Skeletons first; no layout shift
Server time and contention	0.50 s	BFF and backend processing, a busy VPN
First visit	**≈ 1.9 s**	**0.6 s margin for VDI; repeat visits ≈ 1.0 s (service worker)**

Surface 2 follows the same idea: the panels load first and the 3D code after. In Part B the first download is **≈ 34 KB gzipped**; three.js and React Three Fiber (≈ 282 KB) load lazily, and LOD2 models (≈ 0.8 MB) make the room visible within a second after that.

11.2 The 3D frame budget (Surface 2)

Budget (from the brief)	Design target	How
≥ 50 fps at 1080p on integrated graphics	≤ 20 ms per frame: CPU ≤ 8 ms, GPU ≤ 10 ms	React is not in the frame loop: the store updates only on commits; drags touch one instance matrix; render on demand when idle; device pixel ratio capped at 1.25 on integrated GPUs; no real-time shadows
< 300 draw calls	≤ 260 at 2,000 items	Instancing by model × material × LOD (Section 9.4)
< 1 GB browser memory	≤ 600 MB	Compressed textures on the GPU, shared geometry, LOD eviction, bounded history

11.3 Measured in Part B

Metric	Result
Frame rate	**60 fps (16.6 ms) at 500 items, 59 fps (17.1 ms) at 2,500**, at 1920 × 1080 on Intel Iris Xe (Core i7-1255U, 16 GB, Chrome 155), production build
Draw calls · memory	15 at any item count (19 with a selection). 2,000 items: tab **175 MB**, GPU process ≤ 733 MB (shared): under 1 GB
Clearance check	4.8 ms at 2,000 items (realistic layout); 5.3 ms at 500 random overlapping items

First download	≈ 34 KB JS + 3 KB CSS gzipped; 3D code ≈ 282 KB loaded after
Model optimisation	25.8 MB glTF (CC0) → **496 KB**; GPU textures **256** → **3 MB**; ready **1,557** → **321 ms**; 2,500 items **46** → **52 fps**. Instancing off → on, 2,000 items: **≈ 40** → **60 fps**, **6,006** → **15** draw calls

11.4 Measured in CI and in the field

Check	What it asserts
Bundle size limits	`size-limit` per route entry and package; over budget fails the build
Lighthouse CI	Key screens in EN, VI and 中文 on a throttled profile (10 Mbps, 150 ms, 4× CPU slowdown): LCP, a `first-useful` mark, CLS
3D frame time (nightly)	Playwright drives the editor on a real integrated-GPU laptop (headless rendering is not representative): the fixed 2,000-item layout, a scripted orbit and drag; asserts p95 frame time ≤ 20 ms, draw calls ≤ 300, heap ≤ 600 MB
Asset budgets	The asset pipeline fails a model that exceeds its triangle, size or material budget (Section 9.1)
Field data	Web Vitals by route, locale and region; frame time, draw calls and memory by device class from Surface 2

11.5 Error reporting

- **Sentry** with release versions and private source maps; user ids hashed; form values and tokens scrubbed.
- A correlation id on every request also tags the Sentry event, linking a user's error to backend logs.
- Error boundaries per route, and around the 3D view, which falls back to its panels (as in Part B); a lost WebGL context is recorded and recovered.
- Alerts on error rate, slow routes and slow frames per release.

11.6 Testing strategy

Level	Tools	What it covers
Unit	Vitest	Formatters, Zod schemas, event handlers; Surface 2's whole domain: snapping, rules (vs brute force), commands, history, BOM, nudging, group edits, paste (Part B: 47 tests)
Component	Testing Library, Storybook, axe	Every shared component in 3 languages, light/dark and both densities
Contract	OpenAPI types, MSW, zod	API mocks generated from the spec; the host contract tested against a test host page
End-to-end	Playwright	The five main flows by keyboard in 3 locales; offline and slow networks; WebSocket resume; concurrent approval; the embedded editor with token refresh
Visual	Playwright screenshots	Key screens in EN, VI, 中文 and a pseudo-locale; the 3D view with the fixed test layout
Performance	size-limit, Lighthouse CI, frame-time runs	Section 11.4
Manual	NVDA + Edge, TalkBack	Screen-reader pass on the main flows before each release

12 End-to-end sequence: fix a clearance violation

A sales engineer drags a cabinet, the rule check flags a clearance violation, the engineer asks the assistant to fix it, previews the fix and accepts it.

#	Who → whom	Channel	What happens
1	Engineer → 3D view	pointer	Presses on cabinet C-03. It is selected in 3D, in the list and in properties (one store field). Pointer capture starts the drag
2	3D view → domain	in memory	On each move the position snaps to the grid in local preview state; the rule check runs on the preview (≈ 5 ms at 2,000 items)
3	Domain → 3D view, panels	in memory	C-03 comes within 0.4 m of drive D-07: both turn red *while dragging*; the clearance zone outline turns red
4	Engineer → store	pointer up	One `move` command is committed: history gets one step; the rule panel says "C-03 and D-07 are 0.4 m apart"; the live region announces it
5	Store → Configurator API	REST ⑩	`POST /layouts/er-104/ops` with `If-Match: 40` → 200, version 41. The server re-runs the rules and confirms the violation over WSS ⑪
6	Engineer → assistant	REST ⑩	"Fix the clearance issue." `POST /assistant/messages` with references: layout er-104 v41, selection C-03, violation c03|d07 → 202 runId
7	Configurator API → AI agent → LLM	REST ⑭ (server-side)	The agent reads the layout and rules with the engineer's permissions and computes a move that clears 0.6 m
8	API → assistant panel	WSS ⑪	`assistant.delta` tokens stream the explanation; `assistant.done` carries the proposal: move C-03 to (4.9, 2.3), based on v41
9	Assistant → domain	in memory	The client validates and snaps the change, applies it to a copy and runs the rule check: "resolves 1 issue · creates 0"
10	Domain → 3D view	in memory	**Preview:** the ghost layer shows C-03 translucent at its new place with an arrow from the old one; the real layout is unchanged
11	Engineer → store	click / Enter	**Accept:** one batch command "Assistant: move C-03" is applied; both items turn back to normal; "Done: assistant fix" is announced
12	Store → Configurator API	REST ⑩ · WSS ⑪	Operations sent with `If-Match: 41` → v42; the server confirms no violations. If the version had moved on, the proposal would be re-validated first (Section 10.3)
13	Engineer → store	`Ctrl+Z`	(Optional) one undo reverts the whole fix, sent to the server as the inverse operations
—	Throughout	postMessage ⑨	If the token nears expiry, the host bridge refreshes it without interrupting the flow; `app:dirty` is false once v42 is saved

13 Decision log and rejected trade-offs

#	Area	Choice	Why	Rejected, and why
D1	Repository	Nx + pnpm monorepo, versioned packages	Shared code with independent releases	Separate repos (drift); one app for both surfaces (one release train)
D2	Build	Vite	Fast, standard, simple	Webpack / rspack (only needed for Module Federation, kept as the embedding fallback)
D3	Rendering	Client-side SPA, route splitting, service-worker shell	Behind sign-in; a small shell meets 2.5 s	Next.js / SSR (extra servers for little gain); SSR is also meaningless inside an iframe editor
D4	Routing	TanStack Router (Surface 1); in-memory routing (Surface 2)	Typed URL state; the host owns Surface 2's URL	React Router (untyped search params)
D5	Server data	TanStack Query	Caching, retries, offline pause, optimistic rollback	Hand-written fetch; RTK Query (adds Redux); SWR (weaker mutations)
D6	Client state	Zustand	Small, no provider, selective subscriptions, usable outside React (the 3D loop)	Redux Toolkit (more ceremony than needed); Context (re-renders)
D7	Live updates	Native WebSocket: sequence numbers, resume, backoff with jitter	Fixed by the brief	Socket.IO (protocol fixed); polling
D8	Large grids	AG Grid Enterprise, server-side row model	200k rows, pinning, keyboard, bulk selection built in	TanStack Table + Virtual (we would build all of it); MUI DataGrid
D9	Forms	React Hook Form + Zod	Fast long forms; one schema for validation and types	Formik (slower on large forms)
D10	API contract	Types and mocks from OpenAPI	Frontend and backend cannot drift	Hand-written types
D11	Drafts	IndexedDB at once + server autosave	Nothing lost; resume on any device	localStorage only
D12	UI library	Own `@cloudworx/ui`: React Aria + CSS Modules + CSS variables	Light, accessible, re-themeable for a host at runtime	MUI / Ant Design (heavy, hard to re-theme); runtime CSS-in-JS; Tailwind (couples app configs)
D13	Languages	i18next + ICU + `Intl`, lazy per screen	One build, plurals, small downloads	react-intl; per-language builds
D14	Charts	Apache ECharts, lazy-loaded	Canvas speed, CJK support	Any chart library in the main bundle

D15	AI streaming	Start/stop over REST; tokens over the existing WebSocket	Only the two fixed channels	SSE (a third protocol); browser → LLM (leaks keys and data)
D16	Sign-in (Surface 1)	OIDC via a thin BFF; HttpOnly cookie	No tokens in browser JavaScript	Tokens in localStorage (XSS)
D17	Testing	Vitest, Testing Library, Playwright, axe, MSW	A full pyramid incl. accessibility and languages	Manual-only checks
D18	Performance	Lighthouse CI, size limits, real-hardware frame-time runs, field data	Budgets checked on every change	Measuring before release only; headless GPU emulation
D19	Errors	Sentry + correlation ids	One trace from the user to backend logs	Console logs only
D20	Embedding	Iframe + typed `postMessage` contract	Isolation, crash safety, independent releases, security	Micro-frontend (shared runtime; kept as fallback over the same contract)
D21	Sign-in (Surface 2)	Host-issued, audience-scoped bearer token in memory	The host owns sign-in; third-party cookies are restricted	Cookies (blocked in iframes); a second login
D22	3D stack	three.js + React Three Fiber	The scene is a function of the same store as the panels; huge ecosystem; small	Babylon.js (capable, but heavier and its own UI model); raw three.js (more glue code)
D23	Edit model	Commands with apply/revert in a pure-TS domain	Exact undo, batches, server operations, testable	Snapshot undo (memory-heavy at 2,000 items, no operation log)
D24	Rule check	Spatial hash, live on drag; server authoritative	4.8 ms at 2,000 items	All pairs (342 ms); server-only (no live feedback)
D25	Geometry compression	Meshopt + quantisation	Fast WASM decode on weak CPUs, small decoder	Draco (slower decode, larger decoder); none
D26	Textures	KTX2 (Basis)	Stays compressed on the GPU	PNG / JPEG (4–6× GPU memory)
D27	Drawing items	InstancedMesh per model × material × LOD	Draw calls scale with distinct models	One mesh per item (2,000+ calls); one merged static mesh (rebuild per edit)
D28	AI changes	Proposal as commands → ghost preview → one-step accept and undo	The user stays in control; the same rules validate it	Agent edits the layout directly
D29	Mobile	Same responsive web app	Phones need only inbox, uploads, quotes (A10); one codebase	Native app (second codebase, store reviews)
D30	Release	Flags + staged rollout, auto-rollback	A bad release reaches few users	Big-bang releases

14 Assumptions

#	Assumption	If it is wrong
A1	About 2,500 supplier and 500 buyer/approver users; ~1,500 online at once	The gateways scale horizontally; nothing in the client changes
A2	Recent Chrome or Edge on desktop; Chrome on Android for suppliers; WebGL 2 available on the engineers' laptops	Older browsers need polyfills; without WebGL the editor falls back to its panels
A3	Backends provide OpenAPI specs, server-side filter/sort with locale collation, ETags, idempotency keys, a jobs API, and sequence numbers on WebSocket events	The BFF / Configurator API adds what is missing
A4	Surface 1 signs in through the company's OIDC provider	Any standards-based provider fits the same BFF flow
A5	The frontend team may run a thin BFF for Surface 1	**Fallback:** SPA with OIDC + PKCE, tokens in memory, short-lived access tokens with refresh rotation, per-user event filtering in the backend. Workable, but weaker against XSS and slower on the VPN
A6	Corporate policy allows service workers, IndexedDB and the Cache API	The apps still work; repeat visits are slower
A7	An AG Grid Enterprise licence is acceptable	TanStack Table + Virtual with the same server API; ~4–6 extra weeks
A8	~10 Mbps and ~150 ms round trips over VPN; WebSockets pass on port 443	If WebSockets are blocked, the active screen refetches every 30 s and says so
A9	The LLM provider is contracted for no data retention, in an approved region	The assistant is disabled per region by feature flag
A10	On phones, suppliers need the inbox, onboarding, uploads and quotes; grids are desktop-first	Mobile grids become read-only card lists
A11	The host can embed an iframe and issue an OIDC access token for the Configurator API audience	Web-component fallback (Section 7.1); token exchange via the host's backend
A12	A typical room uses at most ~40 of the 300 models	Draw calls grow with distinct models; beyond ~60, merge materials into atlases or use `BatchedMesh`
A13	CAD exports are available as glTF or STEP with real units and a sensible origin	A per-vendor fix-up step in the pipeline
A14	"Integrated graphics" means Intel Iris Xe / UHD 620 class or better	Older GPUs get a lower LOD bias and DPR 1.0 by device class
A15	One engineer edits a layout at a time in v1; collaborators see changes live but do not co-edit	Operations already carry versions, so co-editing can rebase later
A16	Vietnamese and Chinese text in the wireframes is a placeholder; product strings go through professional translation	Translation review is added to each release's checklist

15 Risks and open questions

Risk	Likelihood	Mitigation and early signal
CAD exports far above budget (internal parts, millions of triangles)	High	Pipeline budgets fail the build per model; a per-vendor clean-up profile; the asset dashboard shows models over budget
Integrated GPUs or VDI without GPU acceleration	Medium	Device-class LOD bias and DPR; nightly frame-time runs on real hardware; WebGL fallback to panels
A host forbids iframes or blocks WebAssembly in its CSP	Low	Web-component fallback over the same contract; agree the CSP in onboarding
WebGL context loss on low memory	Medium	Handle `webglcontextlost`, reload GPU resources, keep the layout safe in the store; tracked in Sentry
Chinese font size hurts first load for zh-CN users	Medium	Subset fonts after first paint; Web Vitals per locale
Server collation and accent-insensitive search not available	Medium	Agree the API early; a search index with ICU behind the grid service
Translation quality and late strings	High	Pseudo-locale in CI, translation tool synced on merge, English fallback reported
BFF grows into a "mini monolith"	Medium	No business logic in the BFF; owned by the frontend team with clear limits

Open questions for the team

1. Is a frontend-owned BFF acceptable for Surface 1? (A5)
2. Which hosts will embed Surface 2 first, and can they issue tokens for our API audience? (A11)
3. Which assistant proposals may be accepted with one click, and which must always be manual?
4. Is co-editing of one layout needed in the first year? (A15)
5. Is there a data-residency requirement for China affecting the CDN, Sentry or the LLM region?

A Appendix

A.1 Part B: the reference build

Repository: github.com/Pavithravignesh/3d-layout-editor · **Live demo:** 3d-layout-editor.vercel.app (React 18, TypeScript strict, three.js via React Three Fiber, Zustand). It implements the core of Surface 2 and is where several numbers in this document come from.

Section	Where it is in Part B
Section 8.1 Scene · Section 9.4 instancing	`src/scene/`: floor, `ItemsLayer` (one InstancedMesh per part per type), non-raycast overlays, placement ghost
Section 8.3 Snapping · Section 8.4 commands · Section 8.5 rules	`src/domain/`: `geometry.ts`, `commands.ts`, `history.ts`, `rules.ts` (spatial hash), `group.ts` (group moves, paste); 49 unit tests in total
Section 9 Asset optimisation	`tools/optimise-model.mjs`: simplify, KTX2, Meshopt, before/after report; `src/scene/cabinetModel.ts` loads and times the result
Section 8.6 Sync	`src/state/editorStore.ts`: one store read by the 3D view, list, properties and rule panel
Section 3.3 Tokens and components	`src/design-system/`: a small version of `@cloudworx/ui`
Section 6 Localisation and accessibility	EN/VI with `Intl` numbers; list as one Tab stop with arrow keys; live region for edits, undo and redo
Section 11 Performance	Lazy 3D chunk, vendor chunks, FPS / draw-call readout, repeatable 500-item test layout

Using the editor

Keys	What they do
`Ctrl+Z` · `Ctrl+Y` (or `Ctrl+Shift+Z`)	Undo · redo
Arrow keys (`Shift`: 1 m)	Move the selection one 0.5 m cell
`R` · `Delete`	Rotate the selection 90° · delete it
`Ctrl+A` · `Ctrl+C` / `X` / `V`	Select all · copy, cut, paste
`Esc`	Stop placing, or clear the selection

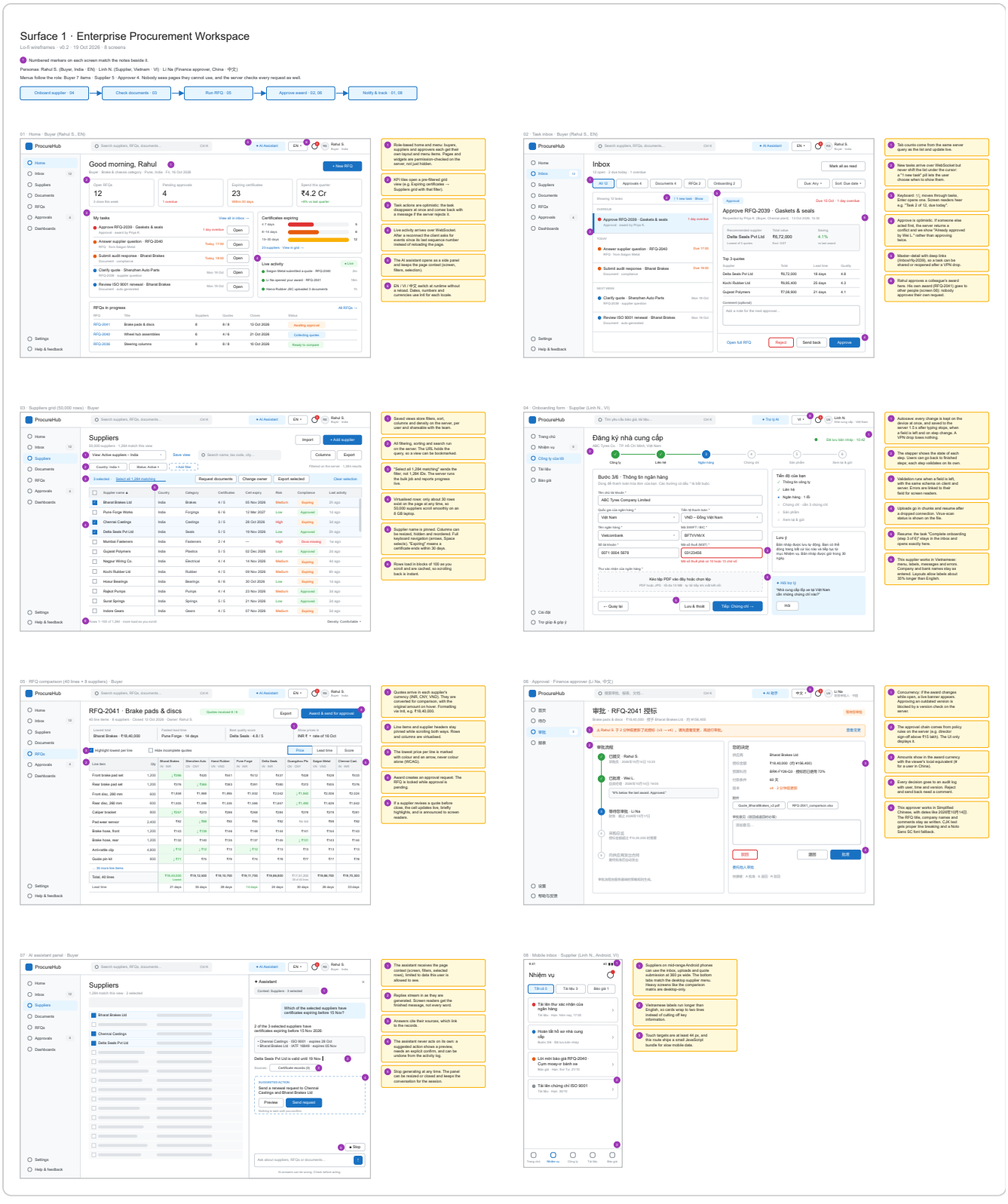
Several items: `Ctrl`-click items in 3D (in the list, `Ctrl`-click adds and `Shift`-click selects a range). Dragging any selected item moves the group; arrow keys, `R` and `Delete` act on all of it. Pasted copies land one cell down-right with the next labels. Each group action is one undo step.

Display switches (top right of the 3D view; they change only the drawing, never the layout): **Boxes / Original / Optimised** draws control cabinets as simple boxes, the 25.8 MB source model or the optimised one; load size, GPU memory and load time appear at the bottom left. **Original** is available only on a local copy after `npm run model:fetch`, as the source is kept out of git and the live demo. **Instancing On / Off** draws items of one kind together (15 draw calls) or each on its own, to show what instancing saves.

A.2 Surface 1 wireframes

01 Home · 02 Task inbox · 03 Suppliers grid · 04 Onboarding (Vietnamese) · 05 RFQ comparison · 06 Approval (Chinese) · 07 AI assistant · 08 Mobile inbox. Menus follow the role (buyer, supplier, approver). Source: `design/wireframes/surface-1-`

`procurement.excalidraw`.



A.3 Glossary

Term	Meaning
BFF	Backend-for-frontend: a thin server for one frontend that handles sign-in, aggregation and filtering
CJK	Chinese, Japanese and Korean, with special font and line-breaking needs

CSP	Content Security Policy: a header listing what a page may load and who may embed it
Design token	A named design value (colour, spacing, font), exposed as a CSS variable
Draw call	One instruction from the CPU to the GPU to draw a batch of geometry
ETag / If-Match	A version stamp used to reject edits based on an outdated version
Idempotency key	A unique id on a command so a retry is not executed twice
Instancing	Drawing many copies of one model in a single draw call
KTX2 / Basis	A texture format that stays compressed in GPU memory
LOD	Level of detail: simpler versions of a model for when it is far away
Meshopt	A glTF geometry compression format with a fast WebAssembly decoder
postMessage	The browser API for messages between a page and an iframe
SSRM	AG Grid's server-side row model, which loads rows in blocks on demand
Spatial hash	A grid of buckets so each item is only compared with its neighbours

A.4 Use of AI tools

I used **Claude Code** (Anthropic, Claude Opus model) as an agent throughout: it proposed plans with options, and nothing was built or changed until I approved it. **I made the decisions** (architecture, stack, scope, trade-offs); the assistant drafted this document's sections from them, generated the editable diagrams and wireframes, wrote Part B's code and tests, and checked the work against the brief. **I reviewed every section, tested every feature by hand, and made every measurement** quoted here. Repeated word-by-word audits against the brief found gaps (the role-based home, wiring, the before/after frame time), which were then added. The README of Part B describes the workflow, example prompts and corrections in full.